Embedded Systems

CS429 Lab9

Name: Dipean Dasgupta **ID:202151188**

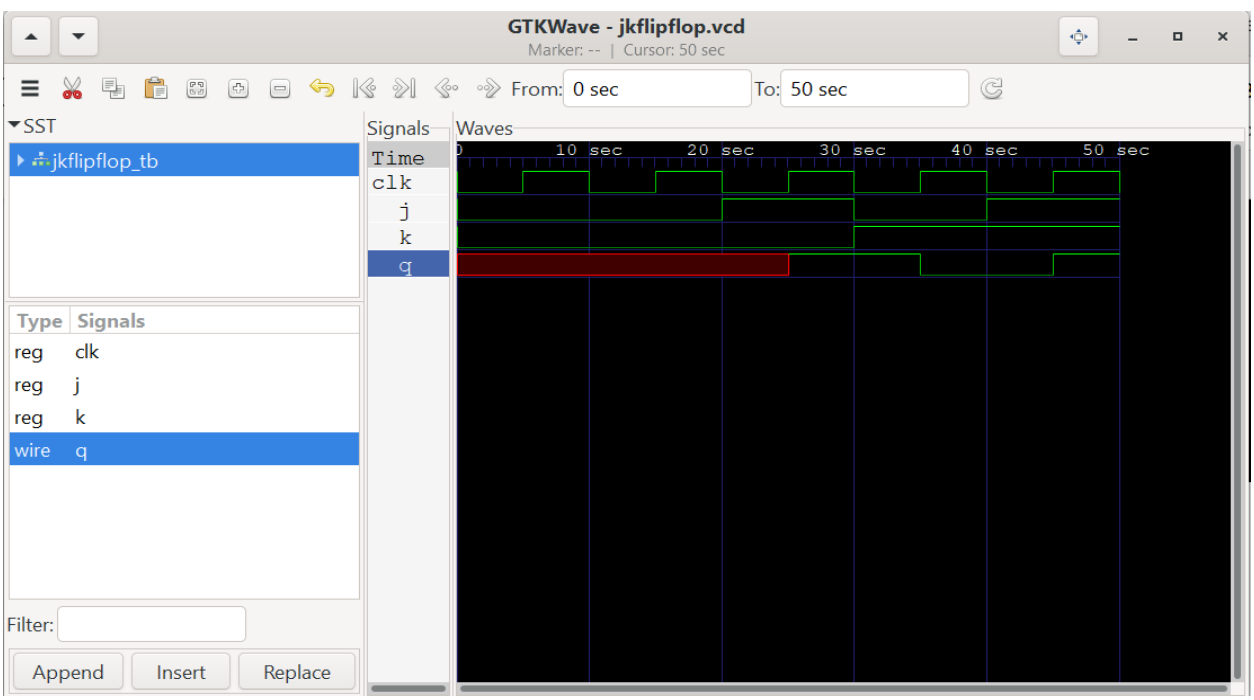
TASK1: Write and verify HDL of JK flip-flop using behavioral description in Verilog.

Code:

```
module jkflipflop(
    input j,
    input k,
    input clk,
    output reg q
    );

    always @(posedge clk)
    begin
        if (j == 1 && k == 0)
            q <= 1;
        else if (j == 0 && k == 1)
            q <= 0;
        else if (j == 1 && k == 1)
            q <= ~q;
    end
endmodule

//testbench
module jkflipflop_tb();
  reg j, k, clk;
  wire q;

  jkflipflop uut (
    .j(j),
    .k(k),
    .clk(clk),
    .q(q)
  );
  initial begin
    j = 0;
    k = 0;
    $dumpfile("jkflipflop.vcd");
    $dumpvars(0, jkflipflop_tb);
```

```
  end
  initial begin
    clk = 0;
    forever #5 clk = ~clk;
  end
  initial begin
    #10;
    j = 0;
    k = 0;
    #10;
    j = 1;
    k = 0;
    #10;
    j = 0;
    k = 1;
    #10;
    j = 1;
    k = 1;
    #10; $finish;
  end
endmodule
```

OUTPUT:



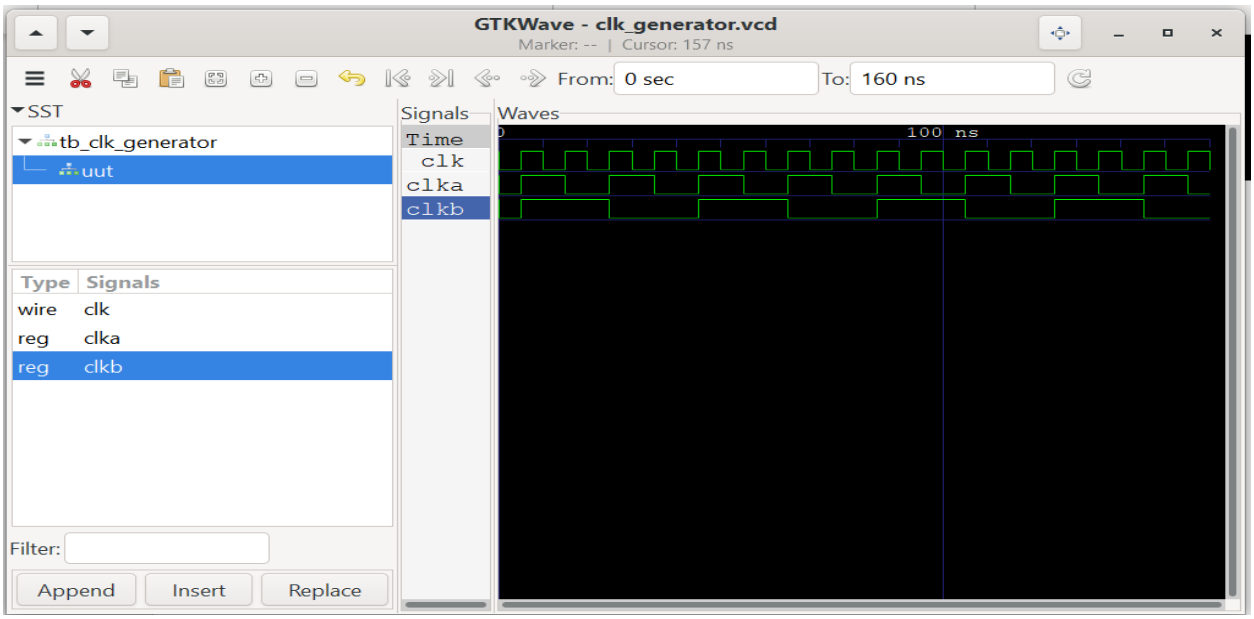
TASK2: For an embedded system, different clock pulses are required to run the different connected blocks. Generate a clock pulse (clk) of 10ns. Using this, generate clk/2 and clk/4 signals.

Code:

```
module clk_generator (
    input clk,
    output reg clka, clkb
);

initial begin
    clka = 1'b0;
    clkb = 1'b0;
    $dumpfile("clk_generator.vcd");
    $dumpvars(0, clk_generator);
end
always @(posedge clk)
    clka <= ~clka;
always @(posedge clka)
    clkb <= ~clkb;

endmodule

`timescale 1ns/1ns
module tb_clk_generator;
  reg clk;
  wire clka, clkb;
  clk_generator uut (
    .clk(clk),
    .clka(clka),
    .clkb(clkb)
  );
  initial begin
    clk = 0;
    forever #5 clk = ~clk;
  end
  initial begin
    #10;
    $display("Initial values: clka=%b, clkb=%b ", clka, clkb);
    #20;
    $display("After 20 cycles: clka=%b, clkb=%b ", clka, clkb);
    #30;
    $display("After 30 cycles: clka=%b, clkb=%b ", clka, clkb);
    #40;
    $display("After 40 cycles: clka=%b, clkb=%b ", clka, clkb);
```

```
    #50;
    $display("After 50 cycles: clka=%b, clkb=%b ", clka, clkb);
    #10;
    $finish;
end
endmodule
```

OUTPUT:



TASK3: An embedded system requires a counter to implement the counting task. Design a 4-bit loadable up/down binary counter using if-else statements

Design Code:

```
module UpDownCounter (
  input wire clk,
  input wire rst,
  input wire load,
  input wire up,
  input wire down,
  output reg [3:0] count
);

  always @(posedge clk or posedge rst) begin
    if (rst) begin
      count <= 4'b0000;  // Reset the counter to zero
    end else begin
      if (load) begin
        // Load new value if load signal is asserted
```

```
        count <= in_data;
      end else if (up) begin
        // Increment the counter if up signal is asserted
        count <= count + 1;
      end else if (down) begin
        // Decrement the counter if down signal is asserted
        count <= count - 1;
      end
    end
  end

endmodule
```

TASK4: Implement a counter that counts from 0 to 127 using while, for, repeat, forever loops.

Code:

```
module Counter(
    input wire clk,
    input wire reset,
    output reg [6:0] count
);

always @(posedge clk or posedge reset) begin
    if (reset) begin
        count <= 7'b0000000;
    end else begin
        count <= count + 1;
    end
end
endmodule

module CounterTestbench;
reg clk;
reg reset;
wire [6:0] count;

// Instantiate the counter module
Counter uut(
    .clk(clk),
    .reset(reset),
    .count(count)
);
```

```
initial begin
    clk = 0;
    forever #1 clk = ~clk;
end
initial begin
    reset = 1;
    #10 reset = 0;
    $monitor("Time=%0t, Count=%d", $time, count);
    #254 $finish;
end
endmodule
```

OUTPUT:

```
Time=249, Count=120
Time=251, Count=121
Time=253, Count=122
Time=255, Count=123
Time=257, Count=124
Time=259, Count=125
Time=261, Count=126
Time=263, Count=127
lab10_4.v:43: $finish called at 264 (1s)
[Done] exit with code=0 in 0.332 seconds
```

For

```
module SimpleCounter (
  input wire clk,
  output reg [6:0] counter
);

  // Define a parameter for the counter limit
  parameter COUNTER_LIMIT = 127;

  // Generate loop to create sequential blocks
  genvar i;
  generate
    for (i = 0; i <= COUNTER_LIMIT; i = i + 1) begin : COUNTER_INST
      always @(posedge clk) begin
        if (counter < COUNTER_LIMIT) begin
          counter <= counter + 1;
        end else begin
          counter <= 0;
```

```
        end
      end
    end
  endgenerate
endmodule
```

Repeat

```
module SimpleCounter (
  input wire clk,
  output reg [6:0] counter
);

  always @(posedge clk) begin
    repeat (128) begin
      if (counter < 127) begin
        counter <= counter + 1;
      end else begin
        counter <= 0;
      end
    end
  end

endmodule
```

Forever

```
module SimpleCounter (
  input wire clk,
  output reg [6:0] counter
);

  always @(posedge clk) begin
    forever begin
      if (counter < 127) begin
        counter <= counter + 1;
      end else begin
        counter <= 0;
      end
    end
  end

endmodule
```

TASK5: Consider the number as 16'hABCD, make it as 16'hDCBA using repeat, while and for loops.

Code(While):

```
module while_reverse;
    reg [15:0] number = 16'habcd;
    reg [15:0] reversed_number = 0;
    initial begin
        $dumpfile("reverse_while.vcd");
        $dumpvars(0, while_reverse);
        while(number > 0) begin
            reversed_number = (reversed_number << 4) | (number & 4'hF);
            number = number >> 4;
        end
        $display("Reversed number: %h", reversed_number);
        #10 $finish;
    end
endmodule
```

OUTPUT:

```
[Running] lab10_5.v
VCD info: dumpfile reverse_while.vcd opened for output.
Reversed number: dcba
lab10_5.v:26: $finish called at 10 (1s)
[Done] exit with code=0 in 0.384 seconds
```

Code(for):

```
module for_reverse;
    reg [15:0] number = 16'hABCD;
    reg [15:0] reversed_number = 0;
    integer i;
    initial begin
        $dumpfile("reverse_for.vcd");
        $dumpvars(0, for_reverse);
        for(i = 0; i < 4; i = i + 1) begin
            reversed_number = (reversed_number << 4) | (number & 4'hF);
            number = number >> 4;
        end
        $display("Reversed number: %h", reversed_number);
        #10 $finish;
    end
endmodule
```

Code(repeat):

```
module Repeat_reverse;
    reg [15:0] number = 16'hABCD;
    reg [15:0] reversed_number = 0;
    integer i;
    initial begin
        $dumpfile("reverse_repeat.vcd");
        $dumpvars(0, tb);
        repeat(4) begin
            reversed_number = (reversed_number << 4) | (number & 4'hF);
            number = number >> 4;
        end
        $display("Reversed number: %h", reversed_number);
        #10 $finish;
    end
endmodule
```